

Deep Temporal Learning for HIV Forecasting: Modeling Epidemic Dynamics with Long Short-Term Memory Networks

Daniel Olofin

2025-10-26

Loading of packages

```
library(readxl)
library(readr)
library(keras)
library(tensorflow)
library(viridis)
```

```
## Loading required package: viridisLite
```

```
library(scales)
```

```
##
```

```
## Attaching package: 'scales'
```

```
## The following object is masked from 'package:viridis':
```

```
##
```

```
##   viridis_pal
```

```
## The following object is masked from 'package:readr':
```

```
##
```

```
##   col_factor
```

```
library(tidyr)
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
## v dplyr      1.1.4      v purrr      1.1.0
```

```
## v forcats    1.0.0      v stringr    1.5.1
```

```
## v ggplot2    4.0.0      v tibble     3.3.0
```

```
## v lubridate  1.9.4
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x scales::col_factor() masks readr::col_factor()
```

```
## x purrr::discard()     masks scales::discard()
```

```
## x dplyr::filter()      masks stats::filter()
```

```
## x dplyr::lag()         masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(dplyr)
library(tidyr)
library(lubridate)
library(ggplot2)
library(keras)
library(tensorflow)
library(purrr)
library(abind)
library(patchwork)
```

Import Dataset

```
HiV_data <- read_xlsx("Hivdata.xlsx")
```

Data Cleaning, Feature Engineering and Basics Statistics

Data Visualization

Long Short-Term Memory Modeling

```
# --- Reproducibility ---
set.seed(123)
tensorflow::tf$random$set_seed(123)

# --- 0) Data: ensure date is Date and sorted monthly ---
stopifnot(all(c("YEAR", "Month", "prevalence", "Month_num", "date") %in% names(df)))
df <- df %>%
  mutate(date = as.Date(date)) %>%
  arrange(date)

# Keep only modeling columns
df <- df %>% select(date, prevalence)

# --- Ensure date column and sort ---
df <- df %>%
  mutate(date = as.Date(date)) %>%
  arrange(date) %>%
  select(date, prevalence)

stopifnot(is.numeric(df$prevalence), inherits(df$date, "Date"))
```

Create Differenced Series - Differencing helps to make the time series stationary — good for LSTM training.

```

# Anchor the original (level) series once
Series <- as.numeric(df$prevalence)

# Difference the prevalence
dified <- diff(df$prevalence, differences = 1)

# Replace NA with 0 for first differenced value
dified <- c(0, dified)

```

Apply Your Lag Function

```

lag_transform <- function(x, k = 1) {
  lagged = c(rep(NA, k), x[1:(length(x) - k)])
  DF = as.data.frame(cbind(lagged, x))
  colnames(DF) <- c(paste0("x-", k), "x")
  DF[is.na(DF)] <- 0
  return(DF)
}

```

```

supervised <- lag_transform(dified, 1)
head(supervised)

```

```

##   x-1  x
## 1   0  0
## 2   0  2
## 3   2 -1
## 4  -1 -2
## 5  -2  6
## 6   6 -1

```

Split Dataset into Training & Testing Sets (70/30 Time-Series Split)

```

## Split into train and test sets for time series

N <- nrow(supervised)      # Total observations
n <- round(N * 0.70, digits = 0) # 70% split point

train <- supervised[1:n, ]  # First 70% as training
test  <- supervised[(n+1):N, ] # Remaining 30% as testing

```

Normalize the Data for LSTM

```

## Function to scale data to desired feature range
scale_data <- function(train, test, feature_range = c(-1, 1)) {

  train_min <- min(train)
  train_max <- max(train)

  # Scale train & test using training min/max ONLY
  scaled_train <- ((train - train_min) / (train_max - train_min)) *
    (feature_range[2] - feature_range[1]) + feature_range[1]

  scaled_test <- ((test - train_min) / (train_max - train_min)) *
    (feature_range[2] - feature_range[1]) + feature_range[1]

  # Return scaled sets and scaler values
  return(list(
    scaled_train = scaled_train,
    scaled_test  = scaled_test,
    scaler       = c(min = train_min, max = train_max)
  ))
}

## Apply scaling
Scaled <- scale_data(train, test, c(-1, 1))

# Extract scaled train & test values
x_train <- Scaled$scaled_train[,1] # lagged values
y_train <- Scaled$scaled_train[,2] # current values

x_test  <- Scaled$scaled_test[,1]
y_test  <- Scaled$scaled_test[,2]

```

Correct Inverse-Scaling Function (Matching your [-1,1] scaling)

```

## inverse transform function (scale back to original units)
invert_scaling <- function(scaled, scaler, feature_range = c(-1, 1)){

  min_val <- scaler[1]
  max_val <- scaler[2]

  ## unpack feature range
  fr_min <- feature_range[1]
  fr_max <- feature_range[2]

  # reverse scaling
  inverted <- (((scaled - fr_min) / (fr_max - fr_min)) *
    (max_val - min_val)) + min_val

  return(inverted)
}

```

Modeling

```
# =====  
# Reshape to 3D [samples, timesteps, features]  
# =====  
dim(x_train) <- c(length(x_train), 1, 1)  
dim(x_test)  <- c(length(x_test),  1, 1)  
  
X_shape2 <- dim(x_train)[2] # timesteps = 1  
X_shape3 <- dim(x_train)[3] # features  = 1  
batch_size <- 1             # must divide #samples; 1 is always safe  
units <- 8                  # hidden size (tune as needed)  
  
# =====  
# Build model (stateful LSTM)  
# =====  
model <- keras_model_sequential() %>%  
  layer_lstm(units,  
             batch_input_shape = c(batch_size, X_shape2, X_shape3),  
             stateful = TRUE,  
             return_sequences = FALSE) %>%  
  layer_dense(units = 1)
```

compile

```
# --- Reproducibility ---  
set.seed(123)  
tensorflow::tf$random$set_seed(123)  
  
model %>% compile(  
  loss = "mse",  
  optimizer = optimizer_adam(learning_rate = 0.001),  
  metrics = c("mae")  
)  
  
summary(model)
```

```
## Model: "sequential"  
## -----  
## Layer (type)                Output Shape          Param #  
## =====  
## lstm (LSTM)                 (1, 8)                320  
## dense (Dense)               (1, 1)                9  
## =====  
## Total params: 329 (1.29 KB)  
## Trainable params: 329 (1.29 KB)  
## Non-trainable params: 0 (0.00 Byte)  
## -----
```

Fit the model

```
# --- Reproducibility ---
set.seed(123)
tensorflow::tf$random$set_seed(123)

epochs <- 50

for (e in 1:epochs) {
  model %>% fit(
    x = x_train, y = y_train,
    batch_size = batch_size,
    epochs = 1,
    shuffle = FALSE,
    verbose = 0
  )
  model %>% reset_states()
  if (e %% 10 == 0) cat("Epoch", e, "complete\n")
}
```

```
## Epoch 10 complete
## Epoch 20 complete
## Epoch 30 complete
## Epoch 40 complete
## Epoch 50 complete
```

##Prediction

```
# --- Reproducibility ---
set.seed(123)
tensorflow::tf$random$set_seed(123)

# =====
# Predict (train & test)
# =====
train_pred_scaled <- model %>% predict(x_train, batch_size = batch_size)
```

```
## 101/101 - 0s - 127ms/epoch - 1ms/step
```

```
model %>% reset_states()
test_pred_scaled <- model %>% predict(x_test, batch_size = batch_size)
```

```
## 43/43 - 0s - 14ms/epoch - 318us/step
```

```
# =====
# Invert scaling to original units
# (We used feature_range = c(-1, 1) earlier)
# =====
inv_scale <- function(z_scaled, scaler, feature_range = c(-1, 1)) {
  fr_min <- feature_range[1]; fr_max <- feature_range[2]
```

```

train_min <- scaler["min"]; train_max <- scaler["max"]
((z_scaled - fr_min) / (fr_max - fr_min)) * (train_max - train_min) + train_min
}

y_train_hat <- inv_scale(train_pred_scaled[,1], Scaled$scaler, c(-1, 1))
y_test_hat <- inv_scale(test_pred_scaled[,1], Scaled$scaler, c(-1, 1))

y_train_true <- inv_scale(y_train, Scaled$scaler, c(-1, 1))
y_test_true <- inv_scale(y_test, Scaled$scaler, c(-1, 1))

```

Matrices Evaluation

```

# Evaluate
rmse <- function(a, b) sqrt(mean((a - b)^2))
mae <- function(a, b) mean(abs(a - b))

cat(sprintf("Train RMSE: %.4f | MAE: %.4f\n",
            rmse(y_train_true, y_train_hat),
            mae(y_train_true, y_train_hat)))

```

```
## Train RMSE: 5.8766 | MAE: 4.7388
```

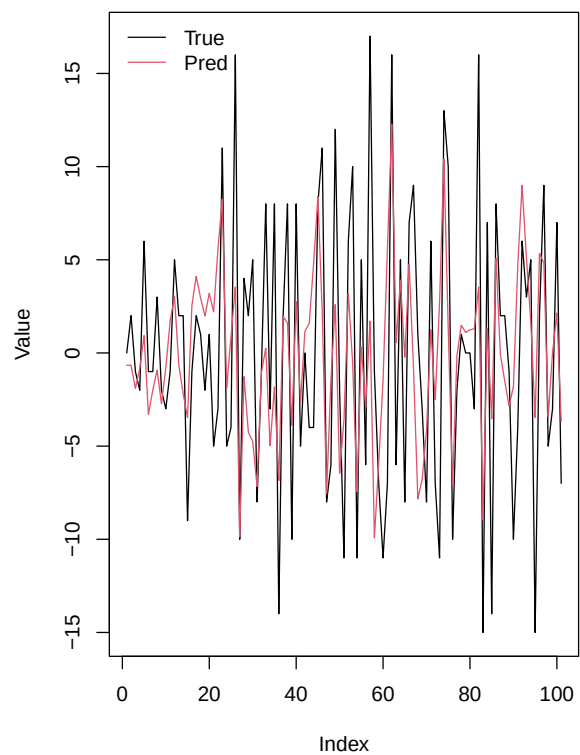
```

cat(sprintf("Test RMSE: %.4f | MAE: %.4f\n",
            rmse(y_test_true, y_test_hat),
            mae(y_test_true, y_test_hat)))

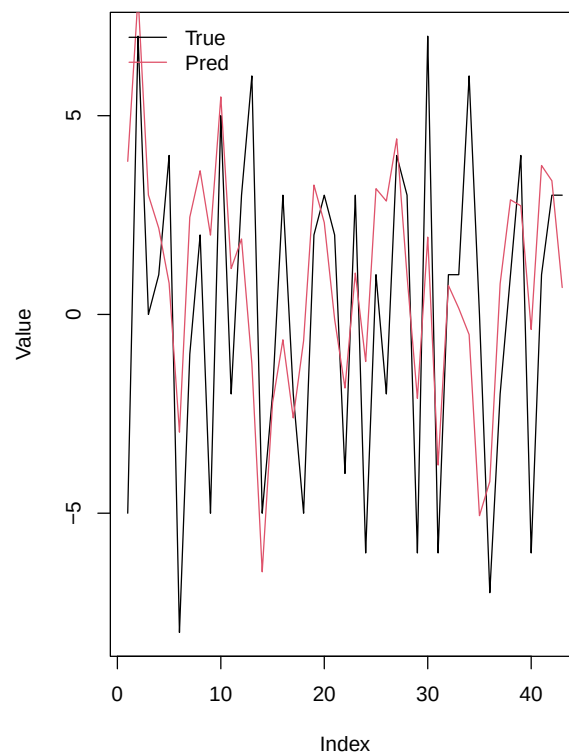
```

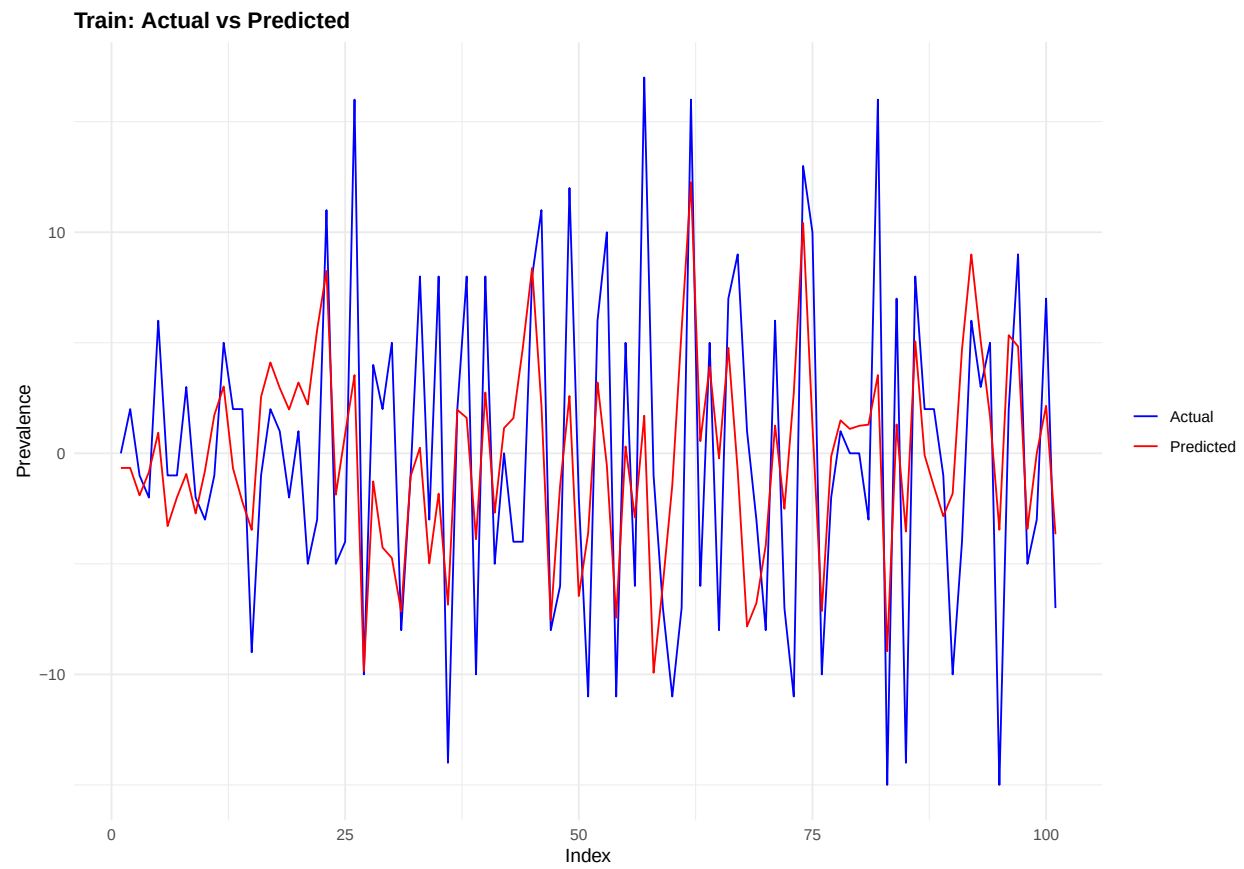
```
## Test RMSE: 3.5363 | MAE: 2.8526
```

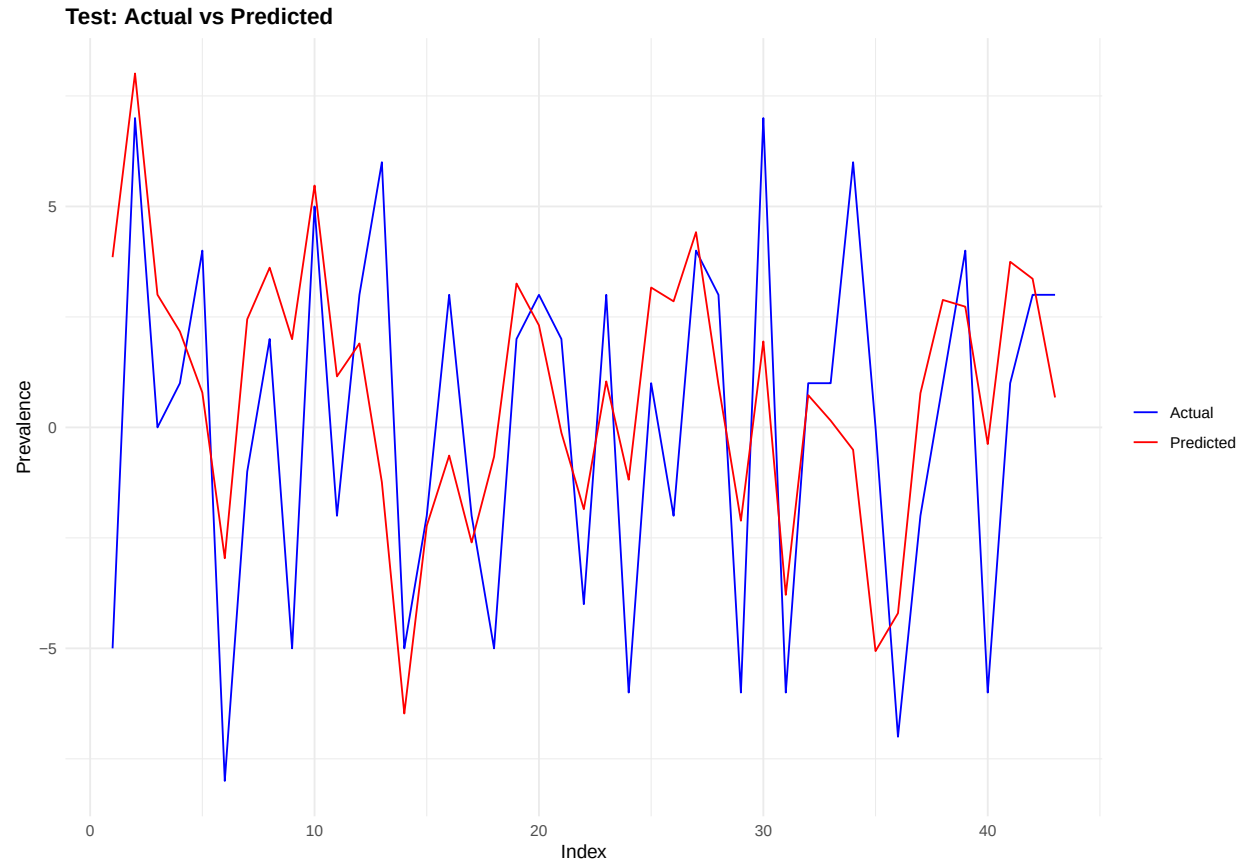
Train: True vs Pred



Test: True vs Pred







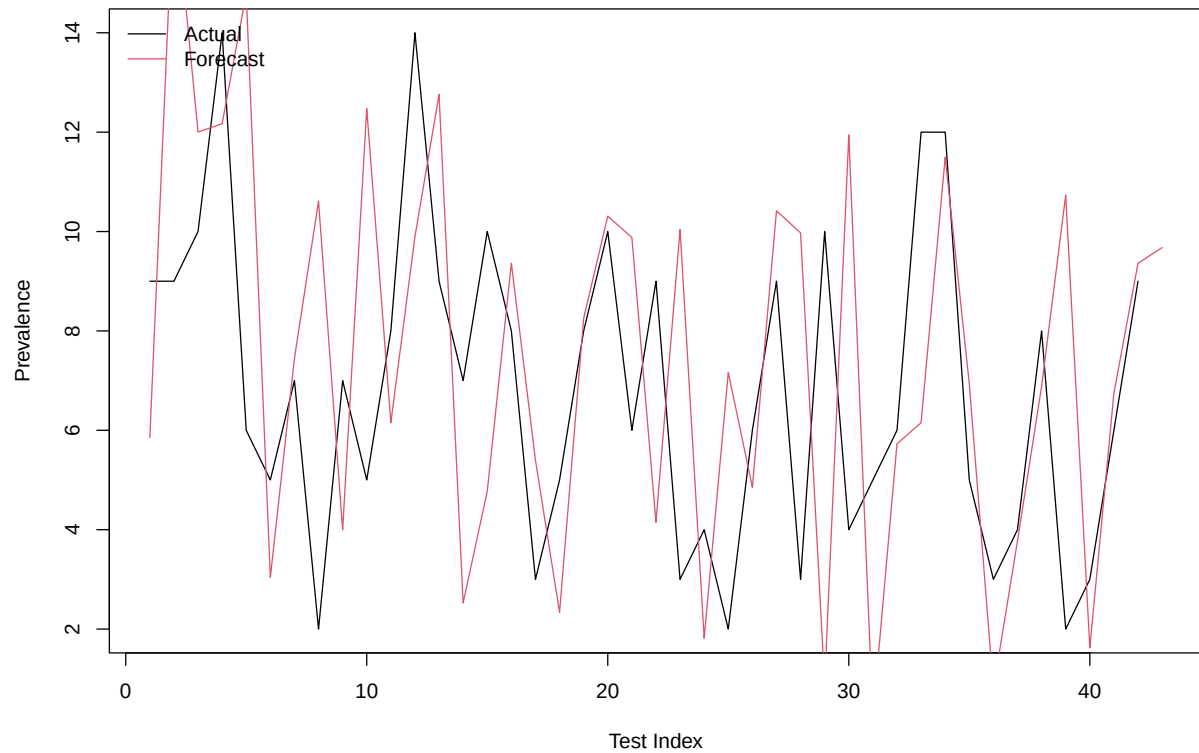
```
## 1/1 - 0s - 44ms/epoch - 44ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
```

```

## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 6ms/epoch - 6ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step

```

Test: Actual vs Reconstructed Forecast (Level)



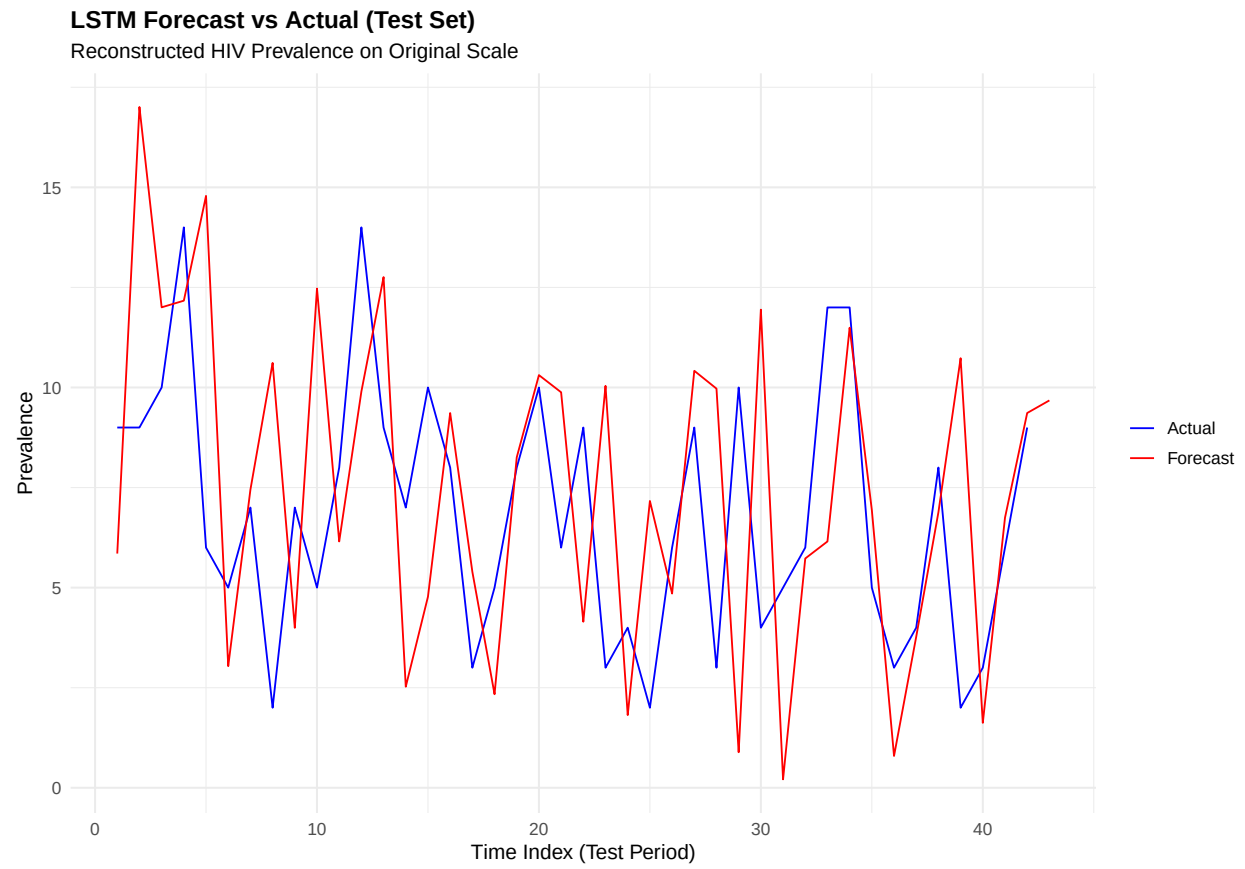


Fig. 1: Data preprocessing pipeline (flowchart)